

Capped Flexible-Docking Ordered-Sortie MILP

Formulation, Written to Match the Implementation

This document writes out the mixed-integer linear program (MILP) that is actually implemented in `model/capped_flexible_docking_ordered_sortie_model.py`. It is not a transcription of the original PDF formulation. Every symbol, set, and constraint below corresponds to named variables, constraints, or helper blocks in the code, so the formulation can be checked against the source directly. Where this model departs from the original PDF formulation, that is called out explicitly in Section 9.

1 Sets and Indices

Symbol	Meaning
$C = \{1, \dots, n\}$	Customers. $n = C $.
$P = \{0, n+1\}$	Depots: 0 is the start depot, $n+1$ is the end depot.
V	$V = \{0, 1, \dots, n+1\} = C \cup P$: all nodes.
$A \subseteq V \times V$	Directed arc set (from <code>arcs.csv</code>); $(i, j) \in A$ means a truck may travel directly from i to j .
$T = \{1, \dots, T \}$	Trucks, $ T = \text{NUM_TRUCKS}$.
$D = \{1, \dots, D \}$	Drones, after lossless fleet trimming (Section 7).
$R = \{1, \dots, R \}$	Robots, after lossless fleet trimming.
S^d	Set of candidate ordered drone sorties (Section 2).
S^r	Set of candidate ordered robot sorties.

2 Ordered Sortie Candidates

Before the MILP is built, `generate_ordered_sorties` enumerates, for every arc $(i, k) \in A$ and every ordered subsequence of customers of length $1, \dots, \text{MAX_CUSTOMERS_PER_SORTIE}$, a candidate *ordered sortie* s with:

- launch node $\iota(s) = i$, recovery node $\kappa(s) = k$;
- ordered customer sequence $\sigma(s) = (\sigma_1(s), \dots, \sigma_{m(s)}(s))$, $\sigma_l(s) \in C$;
- payload $q(s) = \sum_{c \in \sigma(s)} q_c$, kept only if $q(s) \leq$ platform capacity;
- travel distance $\delta(s) = \sum_{(u,v) \in \text{edges}(s)} \text{dist}_{uv}$ (drone: `distance_matrix`; robot: `robot_distance_matrix`), kept only if $\delta(s) \leq$ platform endurance;
- duration $\tau(s) = \sum_{(u,v) \in \text{edges}(s)} \text{time}_{uv}$;
- cost $\gamma(s) = \sum_{(u,v) \in \text{edges}(s)} \text{cost}_{uv}$,

where

$$\text{edges}(s) = ((\iota(s), \sigma_1(s)), (\sigma_1(s), \sigma_2(s)), \dots, (\sigma_{m(s)}(s), \kappa(s))).$$

For each $(i, k) \in A$, only the `TOP_SORTIES_PER_TRUCK_LEG` candidates with the most customers (ties broken by lowest cost, then duration) are kept. S^d uses drone time/distance matrices and (Q_d, E_d) ; S^r uses robot time/distance matrices and (Q_r, E_r) .

Therefore, an “integer optimal solution” from the solver is optimal over the retained capped candidate-sortie sets S^d and S^r . It is not a proof of global optimality over every possible ordered sortie in the unrestricted problem.

3 Parameters

Symbol	Meaning (source)
q_i	Demand of customer $i \in C$ (<code>customers.csv</code>).
$e_i, \bar{e}_i$	Time-window open/close time of customer i (<code>open_time</code> , <code>close_time</code>).
θ_i	Service time of customer i (<code>service_time</code>).
dist_{ij}	Truck/drone travel distance, $i, j \in V$ (<code>distance_matrix</code>).
tt_{ij}	Truck travel time, $i, j \in V$ (<code>truck_time_matrix</code>).
tt_{ij}^p	Truck travel time departing i at the start of period p , $i, j \in V$ (<code>truck_time_matrix_{profile}_p{p}.csv</code> ; Section 8).
dt_{ij}	Drone travel time, $i, j \in V$ (<code>drone_time_matrix</code>).
rdist_{ij}	Robot travel distance, $i, j \in V$ (<code>robot_distance_matrix</code>).
rt_{ij}	Robot travel time, $i, j \in V$ (<code>robot_time_matrix</code>).
Q_t, Q_d, Q_r	Truck / drone / robot payload capacity.
E_d, E_r	Drone / robot endurance, as a maximum route distance per sortie.
$T_{\max}$	Route-duration limit (10 h in the TDRP-TW data).
C_w, C_{veh}	Truck time-cost and distance-cost coefficients.
C_w^d, C_{drone}^d	Drone time-cost and distance-cost coefficients (<code>C_w_drone</code> , <code>C_drone</code>); these are explicit in the processed cases, with code fallbacks only for backward compatibility.
C_w^r, C_{rob}^r	Robot time-cost and distance-cost coefficients (<code>C_w_r</code> , <code>C_rob</code>).
f_t, f_d, f_r	Fixed activation cost per used truck / selected drone sortie / selected robot sortie (<code>truck_fixed_cost</code> , <code>drone_fixed_cost</code> , <code>robot_fixed_cost</code> ; values 30, 10, 8 from Malik et al., VRP-DR, arXiv:2505.23584, Table 3).
λ_T, λ_W	Penalty weights for route-duration excess and time-window lateness.
μ_d^{dep}	Drones a used truck may launch from the depot (<code>DRONES_CARRIED_AT_DEPOT</code>).
μ_d^{trk}	Distinct drones a single truck may use (<code>MAX_DRONES_PER_TRUCK</code>).
μ_r^{dep}	Robots a used truck may launch from the depot (<code>ROBOTS_CARRIED_AT_DEPOT</code>).
μ_r^{trk}	Distinct robots a single truck may use (<code>MAX_ROBOTS_PER_TRUCK</code>).
M	Big-M, $M = \max(T_{\max}, 10 \cdot V \cdot \max(\text{tt}, \text{dt}, \text{rt}), 10000)$.

3.1 Travel-cost coefficients

$$\text{tcost}_{ij} = \text{tt}_{ij} C_w + \text{dist}_{ij} C_{veh}, \quad (i, j) \in A \quad (1)$$

$$\text{dcost}_{ij} = \text{dt}_{ij} C_w^d + \text{dist}_{ij} C_{drone}^d, \quad i, j \in V \quad (2)$$

$$\text{rcost}_{ij} = \text{rt}_{ij} C_w^r + \text{rdist}_{ij} C_{rob}^r, \quad i, j \in V \quad (3)$$

Sortie cost $\gamma(s)$ in Section 2 sums dcost (drone sorties, $s \in S^d$) or rcost (robot sorties, $s \in S^r$) over $\text{edges}(s)$.

4 Decision Variables

Symbol	Code name	Meaning
$x_{v,i,j} \in \{0, 1\}$	<code>x[v,i,j]</code>	Truck v traverses arc $(i, j) \in A$.
$\eta_v \in \{0, 1\}$	<code>used_truck[v]</code>	Truck v is used.
$\text{vis}_{v,i} \in \{0, 1\}$	<code>visit[v,i]</code>	Truck v visits customer $i \in C$.
$u_{v,i} \in [0, n]$	<code>u[v,i]</code>	MTZ rank of customer i on truck v 's route.
$y_{d,s} \in \{0, 1\}$	<code>y[d,sid]</code>	Drone d flies ordered sortie $s \in S^d$.
$h_{v,d,s}^L \in \{0, 1\}$	<code>h_launch[v,d,sid]</code>	Truck v launches drone d for sortie s .
$h_{v,d,s}^R \in \{0, 1\}$	<code>h_recover[v,d,sid]</code>	Truck v recovers drone d for sortie s .
$\xi_{v,d} \in \{0, 1\}$	<code>drone_truck[v,d]</code>	Drone d interacts with truck v (launch and/or recovery).
$z_{r,s} \in \{0, 1\}$	<code>z[r,sid]</code>	Robot r performs ordered sortie $s \in S^r$.

$g_{v,r,s}^L \in \{0,1\}$	<code>g_launch[v,r,sid]</code>	Truck v launches robot r for sortie s .
$g_{v,r,s}^R \in \{0,1\}$	<code>g_recover[v,r,sid]</code>	Truck v recovers robot r for sortie s .
$\zeta_{v,r} \in \{0,1\}$	<code>robot_truck[v,r]</code>	Robot r interacts with truck v .
$a_{v,i}^t \geq 0$	<code>a_t[v,i]</code>	Arrival time of truck v at node $i \in V$.
$a_i^d \geq 0$	<code>a_d[i]</code>	Drone arrival time at node $i \in V$.
$a_i^r \geq 0$	<code>a_r[i]</code>	Robot arrival time at node $i \in V$.
$\ell_i^t \geq 0$	<code>late_t[i]</code>	Truck time-window lateness at customer $i \in C$.
$\ell_i^d \geq 0$	<code>late_d[i]</code>	Drone time-window lateness at customer $i \in C$.
$\ell_i^r \geq 0$	<code>late_r[i]</code>	Robot time-window lateness at customer $i \in C$.
$\rho_v^t \geq 0$	<code>route_late_t[v]</code>	Route-duration excess of truck v over $T_{\max}$.
$\rho^* \geq 0$	<code>route_late_global</code>	Global route-duration excess (objective uses this).

4.1 Derived service indicators

$$\sigma_i^t = \sum_{v \in T} \text{vis}_{v,i}, \quad i \in C \quad (4)$$

$$\sigma_j^d = \sum_{d \in D} \sum_{\substack{s \in S^d \\ j \in \sigma(s)}} y_{d,s}, \quad j \in C \quad (5)$$

$$\sigma_j^r = \sum_{r \in R} \sum_{\substack{s \in S^r \\ j \in \sigma(s)}} z_{r,s}, \quad j \in C \quad (6)$$

Code: `truck_service`, `drone_service`, `robot_service`.

5 Objective Function

5.1 Variable (distance/time) cost

$$\text{VarCost}^t = \sum_{v \in T} \sum_{(i,j) \in A} \text{tcost}_{ij} x_{v,i,j} \quad (7)$$

$$\text{VarCost}^d = \sum_{d \in D} \sum_{s \in S^d} \gamma(s) y_{d,s} \quad (8)$$

$$\text{VarCost}^r = \sum_{r \in R} \sum_{s \in S^r} \gamma(s) z_{r,s} \quad (9)$$

$$\text{VarCost} = \text{VarCost}^t + \text{VarCost}^d + \text{VarCost}^r \quad (10)$$

5.2 Fixed activation cost

$$\text{FixCost}^t = f_t \sum_{v \in T} \eta_v \quad (11)$$

$$\text{FixCost}^d = f_d \sum_{d \in D} \sum_{s \in S^d} y_{d,s} \quad (12)$$

$$\text{FixCost}^r = f_r \sum_{r \in R} \sum_{s \in S^r} z_{r,s} \quad (13)$$

$$\text{FixCost} = \text{FixCost}^t + \text{FixCost}^d + \text{FixCost}^r \quad (14)$$

5.3 Operating cost and penalties

$$\text{OpCost} = \text{VarCost} + \text{FixCost} \quad (15)$$

$$\text{Pen}_T = \lambda_T \rho^* \quad (16)$$

$$\text{Pen}_W = \lambda_W \left(\sum_{i \in C} \ell_i^t + \sum_{i \in C} \ell_i^d + \sum_{i \in C} \ell_i^r \right) \quad (17)$$

$$\text{TotalPen} = \text{Pen}_T + \text{Pen}_W \quad (18)$$

5.4 Objective

$$\min \quad Z = \text{OpCost} + \text{TotalPen} \quad (19)$$

Code: the objective and KPI expression block. Capacity and endurance are hard constraints (Sections 6.2–6.3) and carry no penalty term; only route-duration excess and time-window lateness are penalized.

6 Constraints

6.1 Customer service (assignment)

$$\sigma_i^t + \sigma_i^d + \sigma_i^r = 1, \quad i \in C \quad (20)$$

Code: `customer_service_{i}`. Every customer is served exactly once, by exactly one of truck, drone, or robot.

6.2 Capacity (hard)

$$\sum_{i \in C} q_i \text{vis}_{v,i} \leq Q_t, \quad v \in T \quad (21)$$

$$q(s) y_{d,s} \leq Q_d, \quad d \in D, s \in S^d \quad (22)$$

$$q(s) z_{r,s} \leq Q_r, \quad r \in R, s \in S^r \quad (23)$$

Code: `truck_capacity_{v}`, `drone_capacity_{d}_{sid}`, `robot_capacity_{r}_{sid}`.

6.3 Endurance (hard)

$$\delta(s) y_{d,s} \leq E_d, \quad d \in D, s \in S^d \quad (24)$$

$$\delta(s) z_{r,s} \leq E_r, \quad r \in R, s \in S^r \quad (25)$$

Code: `drone_endurance_{d}_{sid}`, `robot_endurance_{r}_{sid}`. Candidate generation (Section 2) already excludes any sortie with $\delta(s) > E_d$ (or E_r); these constraints restate the limit explicitly in the MILP.

6.4 Drone sortie–truck assignment and synchronization

For every $d \in D, s \in S^d$:

$$\sum_{v \in T} h_{v,d,s}^L = y_{d,s} \quad (26)$$

$$\sum_{v \in T} h_{v,d,s}^R = y_{d,s} \quad (27)$$

and for every $v \in T$:

$$h_{v,d,s}^L \leq \begin{cases} \eta_v, & \iota(s) = 0 \\ \text{vis}_{v,\iota(s)}, & \iota(s) \in C \\ 0, & \text{otherwise} \end{cases} \quad (28)$$

$$h_{v,d,s}^R \leq \begin{cases} \eta_v, & \kappa(s) = n + 1 \\ \text{vis}_{v,\kappa(s)}, & \kappa(s) \in C \\ 0, & \text{otherwise} \end{cases} \quad (29)$$

Code: `assign_selected_drone_sortie_launch/recovery_truck_{d}_{sid}`, `sync_drone_launch/recover_*`. A selected sortie must have exactly one launch truck and one recovery truck, and that truck must physically be at the launch/recovery node (depot use η_v ; customer nodes use $\text{vis}_{v,\cdot}$). The launch and recovery truck need not be the same truck. The “otherwise” branch is a defensive bound that is never active for the node sets produced by the data (launch/recovery nodes are always 0, $n + 1$, or in C).

6.5 Physical-platform consistency

$$\sum_{s \in S^d} y_{d,s} \leq 1, \quad d \in D \quad (30)$$

$$\sum_{s \in S^r} z_{r,s} \leq 1, \quad r \in R \quad (31)$$

Code: `one_sortie_per_drone_{d}`, `one_sortie_per_robot_{r}`. Each physical drone/robot flies at most one sortie, so no inter-sortie sequencing is needed.

6.6 Truck–platform pairing limits

For every $v \in T$:

$$\sum_{d \in D} \xi_{v,d} \leq \mu_d^{trk} \eta_v \quad (32)$$

$$\sum_{s \in S^d} (h_{v,d,s}^L + h_{v,d,s}^R) \leq 2 \xi_{v,d}, \quad d \in D \quad (33)$$

$$\xi_{v,d} \leq \sum_{s \in S^d} (h_{v,d,s}^L + h_{v,d,s}^R), \quad d \in D \quad (34)$$

$$\sum_{r \in R} \zeta_{v,r} \leq \mu_r^{trk} \eta_v \quad (35)$$

$$\sum_{s \in S^r} (g_{v,r,s}^L + g_{v,r,s}^R) \leq 2 \zeta_{v,r}, \quad r \in R \quad (36)$$

$$\zeta_{v,r} \leq \sum_{s \in S^r} (g_{v,r,s}^L + g_{v,r,s}^R), \quad r \in R \quad (37)$$

Code: `max_distinct_drones_assigned_to_truck_{v}`, `drone_truck_pair_upper/lower_{v}_{d}`, and the robot analogues. $\xi_{v,d} = 1$ iff drone d launches and/or is recovered by truck v at least once (at most one launch and one recovery total, by (30), hence the coefficient 2).

6.7 Launch/recovery node capacity limits

For every $v \in T$ and every node $w \in V$ that is a launch node of some $s \in S^d$:

$$\sum_{d \in D} \sum_{\substack{s \in S^d \\ \iota(s)=w}} h_{v,d,s}^L \leq \begin{cases} \mu_d^{dep} \eta_v, & w = 0 \\ \mu_d^{trk} \text{vis}_{v,w}, & w \neq 0 \end{cases} \quad (38)$$

and for every recovery node $w \in V$ that is a recovery node of some $s \in S^d$:

$$\sum_{d \in D} \sum_{\substack{s \in S^d \\ \kappa(s)=w}} h_{v,d,s}^R \leq \begin{cases} \mu_d^{trk} \eta_v, & w = n+1 \\ \mu_d^{trk} \text{vis}_{v,w}, & w \in C \end{cases} \quad (39)$$

with the symmetric pair for robots using $g^L, g^R, \mu_r^{dep}, \mu_r^{trk}$.

Code:

- `depot_drone_launch_limit_{v}` / `node_drone_launch_limit_{v}_{launch}`;
- `depot_drone_recovery_limit_{v}` / `node_drone_recovery_limit_{v}_{recover}`;
- robot analogues.

6.8 Symmetry breaking

$$\sum_{s \in S^d} y_{d+1,s} \leq \sum_{s \in S^d} y_{d,s}, \quad d \in \{1, \dots, |D| - 1\} \quad (40)$$

$$\sum_{s \in S^r} z_{r+1,s} \leq \sum_{s \in S^r} z_{r,s}, \quad r \in \{1, \dots, |R| - 1\} \quad (41)$$

Code: `symmetry_drone_{d_prev}_{d_next}`, `symmetry_robot_{r_prev}_{r_next}`. Platforms of the same type are identical and each flies at most one sortie, so platform $d+1$ may be used only if platform d is already used.

6.9 Drone and robot time windows

$$a_0^d = 0, \quad a_0^r = 0 \quad (42)$$

$$a_i^d \geq e_i \sigma_i^d, \quad i \in C \quad (43)$$

$$a_i^r \geq e_i \sigma_i^r, \quad i \in C \quad (44)$$

$$\ell_i^d \geq a_i^d - \bar{e}_i, \quad i \in C \quad (45)$$

$$\ell_i^r \geq a_i^r - \bar{e}_i, \quad i \in C \quad (46)$$

$$a_i^d \leq \bar{e}_i + \ell_i^d + M(1 - \sigma_i^d), \quad i \in C \quad (47)$$

$$a_i^r \leq \bar{e}_i + \ell_i^r + M(1 - \sigma_i^r), \quad i \in C \quad (48)$$

Code: `start_time_d, start_time_r, open_drone_{i}, open_robot_{i}, late_drone_def_{i}, late_robot_def_{i}, time_window_drone_{i}, time_window_robot_{i}`.

Reporting note. For a customer i with $\sigma_i^d = 0$, (43) reduces to $a_i^d \geq 0$ and, since λ_W drives $\ell_i^d \rightarrow 0$, (47) reduces to $a_i^d \leq \bar{e}_i$. The variable a_i^d then has no effect on the objective, so the solver may set it to either bound of $[0, \bar{e}_i]$ with no physical meaning. The Arrival Times sheet flags such rows as `not_served_value_arbitrary` (same for a_i^r).

6.10 Ordered sortie timing

For $d \in D$, $s \in S^d$ with sequence $\sigma(s) = (\sigma_1, \dots, \sigma_m)$, launch node $\iota(s)$, recovery node $\kappa(s)$, and for every $v \in T$:

$$a_{\sigma_1}^d \geq a_{v, \iota(s)}^t + \text{dt}_{\iota(s), \sigma_1} - M(1 - h_{v, d, s}^L) \quad (49)$$

$$a_{v, \kappa(s)}^t \geq a_{\sigma_m}^d + \theta_{\sigma_m} + \text{dt}_{\sigma_m, \kappa(s)} - M(1 - h_{v, d, s}^R) \quad (50)$$

and for every consecutive pair (σ_l, σ_{l+1}) in the sequence:

$$a_{\sigma_{l+1}}^d \geq a_{\sigma_l}^d + \theta_{\sigma_l} + \text{dt}_{\sigma_l, \sigma_{l+1}} - M(1 - y_{d, s}) \quad (51)$$

The robot analogues replace $a^d, h^L, h^R, y, \text{dt}$ with $a^r, g^L, g^R, z, \text{rt}$:

$$a_{\sigma_1}^r \geq a_{v, \iota(s)}^t + \text{rt}_{\iota(s), \sigma_1} - M(1 - g_{v, r, s}^L) \quad (52)$$

$$a_{v, \kappa(s)}^t \geq a_{\sigma_m}^r + \theta_{\sigma_m} + \text{rt}_{\sigma_m, \kappa(s)} - M(1 - g_{v, r, s}^R) \quad (53)$$

$$a_{\sigma_{l+1}}^r \geq a_{\sigma_l}^r + \theta_{\sigma_l} + \text{rt}_{\sigma_l, \sigma_{l+1}} - M(1 - z_{r, s}) \quad (54)$$

Code: `sortie_time_drone_launch/recover/seq_*`; `sortie_time_robot_launch/recover/seq_*`. If a sortie s is selected ($y_{d, s} = 1$ or $z_{r, s} = 1$), the platform's timeline runs from the launch truck's arrival at $\iota(s)$, through the ordered sequence with service times, to no later than the recovery truck's arrival at $\kappa(s)$.

6.11 Truck routing completeness

For every $v \in T$:

$$\sum_{j: (0, j) \in A} x_{v, 0, j} = \eta_v \quad (55)$$

$$\sum_{i: (i, n+1) \in A} x_{v, i, n+1} = \eta_v \quad (56)$$

For every $v \in T$ and node $\in C$ (flow conservation):

$$\sum_{i: (i, \text{node}) \in A} x_{v, i, \text{node}} = \sum_{j: (\text{node}, j) \in A} x_{v, \text{node}, j} \quad (57)$$

For every $v \in T$, $i \in C$:

$$\text{vis}_{v, i} = \sum_{h: (h, i) \in A} x_{v, h, i} \quad (58)$$

For every $i \in C$:

$$\sigma_i^t = \sum_{v \in T} \text{vis}_{v,i} \quad (59)$$

For every $v \in T$:

$$\sum_{i \in C} \text{vis}_{v,i} \leq n \eta_v \quad (60)$$

$$\eta_v \leq \sum_{i \in C} \text{vis}_{v,i} \quad (61)$$

$$x_{v,i,j} \leq \eta_v, \quad (i, j) \in A \quad (62)$$

Code:

- `op_depot_departure_{v} / op_depot_return_{v}`;
- `op_flow_{v}_{node}`;
- `op_visit_inbound_{v}_{i}`;
- `op_truck_service_visit_{i}`;
- `op_used_visit_activation_{v} / op_used_requires_service_{v} / op_arc_activation_{v}_{i}_{j}`.

These constraints are not written explicitly in the original PDF, which defines $x_{v,i,j}$ but not full route-continuity; they are the standard constraints needed for x to form valid physical truck routes.

6.12 Truck timing and time windows

$$a_{v,0}^t = 0, \quad v \in T \quad (63)$$

$$a_{v,j}^t \geq a_{v,i}^t + \theta_i [i \in C] + \text{tt}_{ij} - M(1 - x_{v,i,j}), \quad v \in T, (i, j) \in A \quad (64)$$

$$a_{v,i}^t \geq e_i - M(1 - \text{vis}_{v,i}), \quad v \in T, i \in C \quad (65)$$

$$\ell_i^t \geq a_{v,i}^t - \bar{e}_i - M(1 - \text{vis}_{v,i}), \quad v \in T, i \in C \quad (66)$$

Code: `op_start_time_truck_{v}`, `op_time_propagation_{v}_{i}_{j}`, `op_open_truck_{v}_{i} / op_late_truck_{v}_{i}`. Here $[i \in C]$ is 1 if $i \in C$ (so θ_i is added) and 0 if $i \in P$ (depots have no service time). Every used truck departs the start depot at clock time 0.

6.13 Route-duration constraints

For every $v \in T$:

$$\rho_v^t \geq a_{v,n+1}^t - T_{\max} - M(1 - \eta_v) \quad (67)$$

$$a_{v,n+1}^t \leq T_{\max} + \rho_v^t + M(1 - \eta_v) \quad (68)$$

$$\rho_v^t \leq M \eta_v \quad (69)$$

$$\rho_v^t \leq \rho^* + M(1 - \eta_v) \quad (70)$$

Code: `op_route_duration_late_truck_{v}`, `op_route_duration_truck_{v}`, `op_route_duration_late_truck_activation_{v}`, `op_route_duration_late_truck_global_bound_{v}`. Because every truck departs at time 0 ((63)), $a_{v,n+1}^t$ is that truck's route duration. ρ^* is forced, by (69)–(70) together with the objective minimizing $\lambda_T \rho^*$, to equal $\max_{v \in T, \eta_v=1} \rho_v^t$, i.e. the worst per-truck route-duration excess over $T_{\max}$.

Relation to drone/robot routes. The original PDF writes the route-duration condition for $m \in \{t, d, r\}$ (trucks, drones, robots separately). This implementation enforces it only through the truck end-depot arrival $a_{v,n+1}^t$. A drone/robot finishing a sortie late forces its recovery truck to wait (via (50)/(53)), which propagates into that truck's $a_{v,n+1}^t$ and hence into ρ_v^t and ρ^* . So drone/robot lateness is captured indirectly through the truck term, not as a separate a_{n+1}^d, a_{n+1}^r penalty. See Section 9.

6.14 MTZ subtour elimination

For every $v \in T$:

$$u_{v,i} \leq n \text{vis}_{v,i}, \quad i \in C \quad (71)$$

$$\begin{aligned}
u_{v,i} &\geq \text{vis}_{v,i}, & i &\in C & (72) \\
u_{v,i} - u_{v,j} + n x_{v,i,j} &\leq n - 1, & i \neq j \in C, (i,j) &\in A & (73)
\end{aligned}$$

Code: `op_mtz_visit_upper/lower_{v}_{i}`, `op_mtz_{v}_{i}_{j}`. Standard Miller–Tucker–Zemlin constraints; not in the PDF, needed to forbid customer-only subtours disconnected from the depot.

7 Lossless Fleet Trimming

Before D and R are fixed, the raw fleet sizes from `parameters.json` (`NUM_DRONES`, `NUM_ROBOTS`) are trimmed to

$$|D| = \max(1, \min(\mu_d^{trk} \cdot |T|, n)) \quad (74)$$

$$|R| = \max(1, \min(\mu_r^{trk} \cdot |T|, n)) \quad (75)$$

Code: the fleet-trimming block. This is lossless: (32) already caps usable drones at $\mu_d^{trk} \cdot |T|$ per type, and (30) plus (20) cap usable drones at n (one sortie serves at least one customer, each customer served once), so any platform beyond $|D|$ (or $|R|$) is provably unused in every feasible solution.

8 Time-Dependent Travel Times

The original PDF formulation assumes time-dependent travel times $t_{ij}(\tau)$. The MILP above is static: every timing constraint uses the free-flow matrix tt_{ij} . The codebase now ships a time-dependent truck layer for a posteriori evaluation of solutions under $t_{ij}(\tau)$ and for time-dependent heuristic route evaluation (`scripts/time_dependent.py`, `scripts/add_td_matrices.py`).

Method. The working day $[0, H]$ is split into P equal periods. Each period p carries a speed factor $f_p \in (0, 1]$ relative to free-flow speed. The travel time for a departure at the start of period p is computed by *integrating speeds* — the classical construction of Ichoua, Gendreau & Semet (EJOR 144(3), 2003): find the smallest T with $\int_{\tau_0}^{\tau_0+T} f(\tau) d\tau = \text{tt}_{ij}$, where τ_0 is the departure time and $f(\cdot)$ is the stepwise speed profile (repeated cyclically past the horizon). Because speeds, not travel times, are the primitive, the FIFO property holds by construction: departing later can never arrive earlier. The generator asserts `verify_fifo` on every build.

Profiles. Speed factors are defined as resolution-independent piecewise segments over day fractions and discretized at period midpoints, so any P can be used:

- **two_peak:** two-peak urban day, maximum slowdown 2:1, within Figliozzi’s (TRE 48(3), 2012) 2.5:1 empirical bound;
- **mild:** small-city congestion (e.g. DeKalb, IL), slowest factor 0.80;
- **flat:** all factors 1.0, the static control.

Scope and files. Only the truck layer is time-dependent. Robot (pedestrian) congestion data does not exist publicly and drone airspace is uncongested, so dt_{ij} and rt_{ij} stay static; this is recorded in the instance provenance. For each profile the generator writes `truck_time_matrix_{profile}_p{p}.csv`, $p = 0, \dots, P - 1$, with entry $[p][i, j] = \text{tt}_{ij}^p$ in hours, and merges provenance (period bounds, speed factors, citations, `td_fifo_verified`) into `parameters.json`. Flags: `--time-periods P`, `--td-profile`; existing instances are retrofitted with `scripts/add_td_matrices.py`.

Status in this model. The MILP of Sections 1–7 optimizes the static problem. The TD matrices are not decision-model inputs; they evaluate fixed routes under $t_{ij}(\tau)$ (e.g. re-timing a proven-optimal static route through peak periods) and will drive time-dependent heuristic search, with the exact static optimum as the baseline for optimality-gap measurement. Optimizing directly under time dependence would require a period-expanded MILP and is deliberately out of scope.

9 Deviations from the Original PDF Formulation

- **Static optimization with time-dependent evaluation.** The PDF assumes $t_{ij}(\tau)$ throughout; this MILP optimizes static tt_{ij} , while Section 8 provides FIFO-safe time-dependent matrices for evaluating solutions and heuristics under $t_{ij}(\tau)$.
- **Endurance and capacity are hard constraints.** The PDF’s Eq. 12 penalizes endurance/capacity violations with $\lambda_E^d, \lambda_E^r, \lambda_Q$ to allow *heuristic exploration of infeasible solutions*. This MILP instead enforces (21)–(25) as hard constraints (and additionally at sortie-generation time), so $\lambda_E^d, \lambda_E^r, \lambda_Q$ are unused.
- **Route-duration penalty is truck-indexed only**, as discussed after (70), rather than summed separately over $m \in \{t, d, r\}$ as in the PDF’s Eq. 8/Eq. 12. Drone/robot lateness still affects ρ^* indirectly through synchronization.
- **Big-M, waiting penalty, and raw cost fields** (`big_M`, `waiting_penalty_weight`, `truck_variable_cost`, `drone_departure_cost`) from the raw TDRP-TW parameters are superseded: M is computed internally (Section 3, parameter table), platform waiting is reported via the Sortie Timing Audit rather than penalized, and travel costs use $C_{veh}/C_{drone}/C_{rob}$ with the fixed-cost terms (11)–(13).
- **Operational routing constraints** ((55)–(73)) are added because the PDF defines $x_{v,i,j}$ but does not write full route-continuity constraints; without them x does not necessarily form a connected truck route from depot to depot.
- **Makespan** is computed after solving from $a_{v,n+1}^t$ and platform finish times for reporting only. It is not a decision variable and is not part of Z , matching the PDF’s objective structure $P_{PDF} = Z + \text{penalty terms}$ (Eq. 12, where P_{PDF} denotes the PDF’s total-cost objective, not the depot set P of Section 1), which also does not include makespan in the objective.

A full parameter-by-parameter audit is in `parameter_usage_audit.csv` and the README’s “Parameter Usage Audit” section.